AI Agent for Generative Modeling

Cassi Chen

1 Introduction and Overview of System Architecture

Generative modeling requires decisions for preprocessing, model selection, hyperparameters, evaluation, and interpretation. Unlike a traditional benchmarking study, this project develops an AI agent that automatically performs generative modeling on real-world datasets with minimal human guidance. Given a dataset and a user-specified objective, the agent determines an appropriate generative modeling workflow, trains suitable models, evaluates the generated data, and recommends how the generated data can be used for downstream analysis, all autonomously.

The overall agent design in this project was inspired by the UNC BIOS 774 course Lecture 9 slides [1]. Specifically, the researcher specifies the dataset and analysis objective, local Python programs perform deterministic computations, and the LLM recovers context, invokes code, and interprets the results against to the task. The researcher launches the workflow through a single command with `run_agent.py`.

As shown in Figure 1, Codex CLI serves as the LLM reasoning agent that invokes code and interprets results. Codex uses `AGENTS.md` and `SKILL.md` files as its primary instructions, where `AGENTS.md` defines the repository-wide workflow while `SKILL.md` files provide guidance for data inspection, objective interpretation, preprocessing, generative modeling methods, and evaluation. Codex uses these skills to determine which methods are appropriate and how interpret results. Local Python programs provide tools for data inspection, preprocessing, model training, synthetic sample generation, and evaluation. These are all deterministic computations and save model configerations, generated samples, metrics, and results. The Python tools support three preprocessing choces: None, feature-wise standardization, and library-size normalization followed by a `log1p` transformation. The supported generative model are variational autoencoders (VAEs), generative adversarial networks (GANs), normalizing flows, and diffusion models. The agent considers the available methods but selects a subset for the observed data. Based on the evaluation results, the agent may perform targeted follow-up analyses before generating a final report.

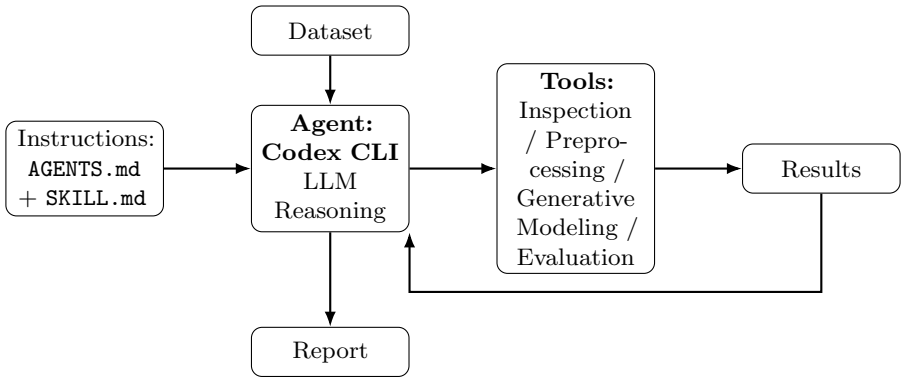


Figure 1: Agent architecture.

2 Agent Decision-Making and Implementation

The agent begins by invoking the data-inspection tool, which describes the number of samples and features, missing values, sparsity, feature types, and other relevant details. Based on these details and the user-specified objective, the agent selects a preprocessing method.

After preprocessing, the agent considers all the available generative modeling methods. The agent selects models based on the results from the data-inspection skill, original data structure, model assumptions, and computational requirements. Hyperparameters are chosen by the agent, with reasonable default values provided. The agent may perform a small targeted follow-up analysis if necessary.

The selected models are fitted using training data only. The agent first performs a short smoke test to verify that the model can train successfully before proceeding with full training. After training, the models generate synthetic samples.

The evaluation data are not used. Model configurations and training histories are saved.

Generated-data evaluations are based on modality-specific and shared diagnostics. An example of a shared diagnostic is mean pairwise distance. Modality-specific diagnostics include feature means for image data and libary size and zero fraction for single-cell RNA data.

The main tools and AI models used in this project are provided in Table 1.

Component	**Type**	**Role**
Codex CLI	AI Model	Tool selection, model decisions, interpretation, and reporting
`inspect_data.py`	Tool	Dataset inspection
`preprocessing.py`	Tool	Data transformations
`train_vae.py`	Tool	VAE training and synthetic sample generation
`train_gan.py`	Tool	GAN training and synthetic sample generation
`train_flow.py`	Tool	Flow-based density modeling and generation
`train_diffusion.py`	Tool	Diffusion training and sample generation
`evaluation_generation.py`	Tool	Synthetic-data diagnostics
`analysis_utils.py`	Tool	General analysis utilities

Table 1: AI model and deterministic Python tools available to the agent.

3 Experimental Results

The system was evaluated on two different datasets: PathMNIST [2] and PBMC3K [3]. Analysis on PathMNIST was conducted using a class-balanced subset of the original dataset, resulting in 4,500 training images (500 per class) and 900 evaluation images (100 per class) across nine classes. For PBMC3K, 2,638 of the original 2,700 cells had corresponding Seurat cell-type annotations and were retained. Of these, 2,110 cells were used for training and 528 for evaluation. Gene filtering reduced the dataset to 2,000 genes. Both runs used `run_agent.py` and executed on CPU because MPS acceleration was unavailable in the Codex CLI sandbox.

In the following subsections, quantitative results focus on comparisons between training and synthetic data. Evaluation data are used as a reference for the nearest-neighbor diagnostics.

3.1 PathMNIST

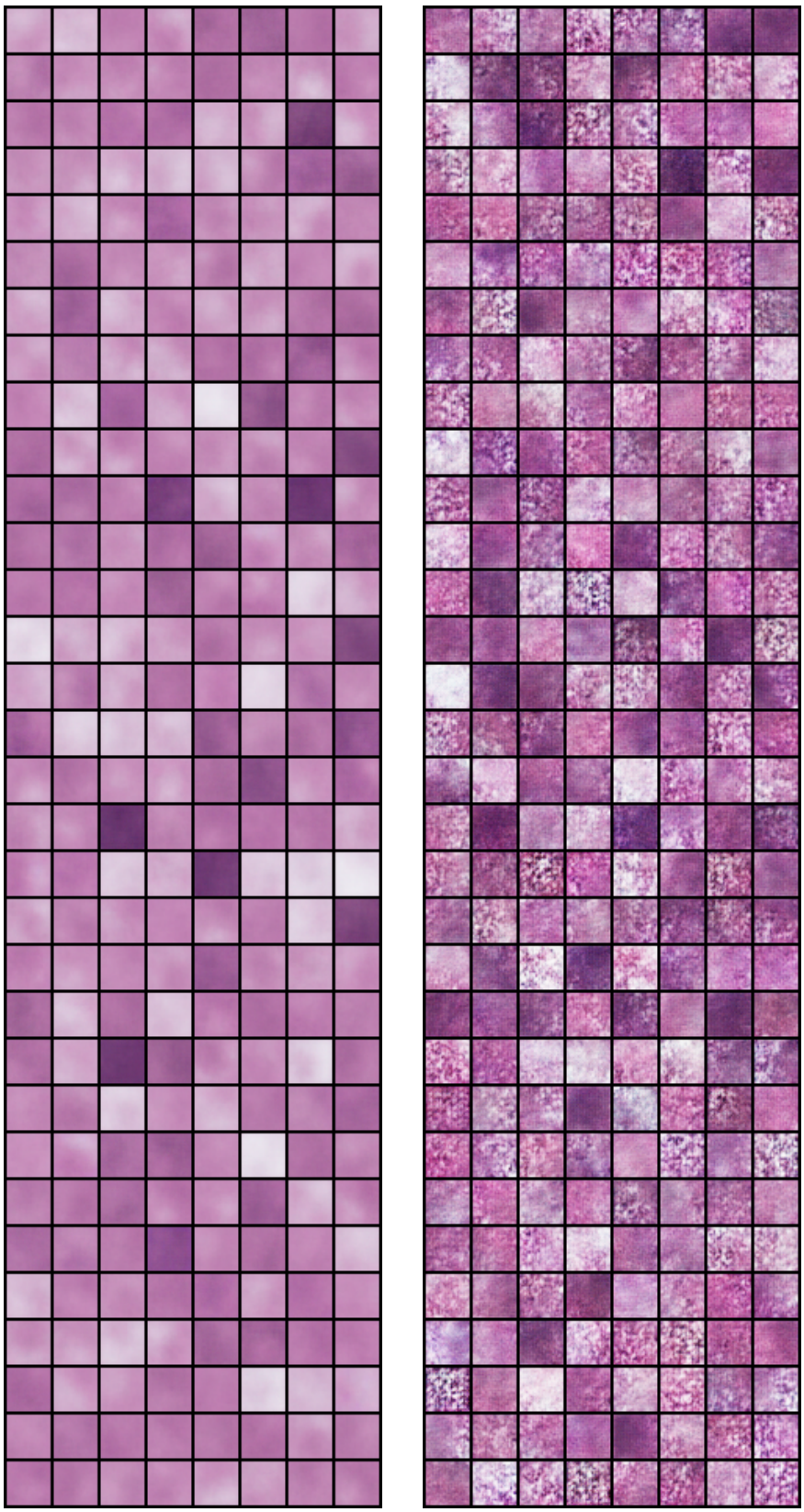
Each 28×28 image in PathMNIST is represented by 2,352 pixel features. The agent selected no additional preprocessing since the generative models performed the appropriate scaling needed already.

The agent selected VAE and GAN image-generation methods. Both models were trained for 30 epochs and generated 256 synthetic images each. Diffusion was not selected because of its computational expense on CPU, and normalizing flow was not selected since it limits a full-image likelihood interpretation.

Table 2 summarizes selected evaluation results. The GAN produced a smaller mean absolute image-feature difference (0.0147) than the VAE (0.0353), achieved a mean pairwise pixel distance (0.1879) closer to the real value (0.1914) than VAE (0.1180), and achieved a nearest-neighbor distance to training (0.0925) closer to the real value (0.0967) than VAE (0.0403). This suggests that the GAN better preserved pixel-level diversity. This is corroborated by Figure 2, where the VAE generally produced blurred textures, while the GAN generated sharper textures.

PathMNIST diagnostic	Real training	VAE	GAN
Mean absolute feature difference	–	0.0353	0.0147
Mean pairwise pixel distance	0.1914	0.1180	0.1879
Nearest-neighbor distance to training	0.0967	0.0403	0.0925

Table 2: Selected evaluation results for PathMNIST.



VAE GAN

Figure 2: VAE (left) and GAN (right) generated PathMNIST images.

The agent recommended `IMPROVE_MODEL`, the results can be improved. Although the GAN performed better on the diagnostics, the model was not validated for histological realism. Thus for downstream uses, the agent advises against

quantitative biological inference, diagnostic use, classifier augmentation, and validation.

3.2 PBMC3K

In contrast to PathMNIST, the agent select selected library-size normalization to 10,000 followed by `log1p` for PBMC3K processing. This is standard preprocessing for scRNA data due to its design nature, where inspection shows sparse nonnegative counts.

The agent then selected a negative-binomial VAE. GANs, normalizing flows, and diffusion models were not selected due to instability or computational requirements.

Table 3 summarizes the evaluation results. The synthetic cells had a similar library size (2023.51) to the training data (2014.86). However, the synthetic cells had more detected genes per cell (625.4205) than the training data (519.9047), a lower zero fraction (0.6873) than the training data (0.7400), a higher mean pairwise profile distance (1.3513) than the training data (1.1027), and a higher nearest-neighbor distance (1.1172) than the training data (0.9402). These differences suggest that the model did not fully reproduce the sparsity and patterns of the original data.

PBMC3K diagnostic	Real training	NB-VAE
Mean library size	2014.8635	2023.5076
Mean detected genes per cell	519.9047	625.4205
Zero fraction	0.7400	0.6873
Mean pairwise profile distance	1.1027	1.3513
Nearest-neighbor distance to training	0.9402	1.1172

Table 3: Selected evaluation results for PBMC3K.

Similar to the PathMNIST reccomendation, the agent recommended `IMPROVE_MODEL`. The agent also stated that the model was `GENERATIVE_APPROPRIATE`, but that although a count-aware generative baseline is justified, it does not indicate biological realism. Thus for downstream use, the model reccomends against using the synthetic data for biological exploration, but states that the synthetic data is still suitable as clearly marked synthetic inputs for software/workflow demonstrations.

4 Strengths, Limitations, and Conclusion

The main strength of the system is that entire process is autonomously done by an agent. The two tests conducted on PathMNIST and PBMC3K also show that the agent is able to adapt across different data modalities.

The system also has many limitations. One is that the agent can only choose among the supplied preprocessing and generative models. Another is that preprocessing and generative model methods and hyperparameter selection rely solely on LLM (Codex CLI) reasoning instead of a specified criteria.

Overall, this project showcases an AI agent that automatically performs generative modeling while maintaining reproducibility.

References

[1] Longfei Zhang. BIOS 774 Course Slides. *An Intro to Building Agents in 2026*, 2026.

[2] Jiancheng Yang, Rui Shi, and Bingbing Ni. MedMNIST v2: A large-scale lightweight benchmark for 2D and 3D biomedical image classification. *Scientific Data*, 10:41, 2023.

[3] 10x Genomics. *PBMC 3K Dataset*. 10x Genomics.